

Khmelnytskyi National University
Department of Information Technonogy

Report

Laboratory work №1

Discipline: “Programming”

Topic: “Linear algorithms”

Completed: 1st year student, group SE-26-1

Nykon Hrytsyshyn
Name, Surname

Checked:

Valeriy Martynyuk
Name, Surname

Khmelnytskyi, 2026

Purpose: To study the structure of the C language program; learn to work with the programming environment of the MS Visual Studio family; get acquainted with I/O operators; to gain skills in using simple data types in the program, compiling the simplest block diagrams of programs and programs in the C language.

Task 1

On the given sides a , b , c of a triangle (fig. 1) to find sizes according to variants. Provide input values a , b , c from the keyboard. Output of initial data to carry out with the corresponding explanatory text.

Develop an algorithm and program for finding the angle β and median m .

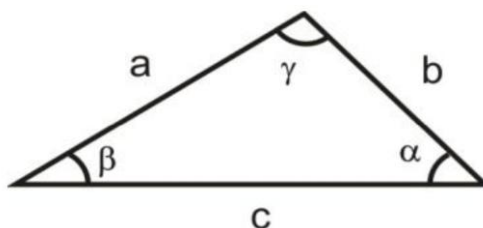


Fig. 1 – Triangle

1.1. Block diagram of the program algorithm

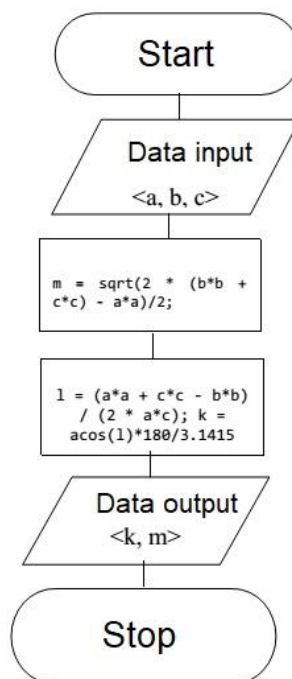


Fig. 2 – Block diagram of the program algorithm

					LW-SE-26-1 09		
Ch.	Sheet	N° document	Signature	Date	Laboratory work N°1		
Developed		Hrytsyshyn N.B.					
Checked		Martynyuk V.V.					
N.control							
Confirmed							
					Letter	Sheet	Sheets
					H	1	16
					KNU group SE-26-1		

1.2. Program text

```
#include "tasks.h"
#include "utils.h"
#include <math.h>
#include <stdio.h>

static double triangle_angle_gamma_deg(
    const double a,
    const double b,
    const double c
) {
    const double cos_gamma = (a * a + b * b - c * c) / (2.0 * a * b);
    const double gamma_rad = acos(cos_gamma);

    return gamma_rad * 180.0 / PI;
}

static double triangle_bisector_gamma(
    const double a,
    const double b,
    const double c
) {
    const double cos_gamma = (a * a + b * b - c * c) / (2.0 * a * b);
    const double gamma_rad = acos(cos_gamma);

    return 2.0 * a * b * cos(gamma_rad / 2.0) / (a + b);
}

void run_task1(void) {
    double a = 0.0;
    double b = 0.0;
    double c = 0.0;

    puts("\n=== Task 1: triangle - angle gamma and bisector w ===\n");

    if (!read_doubles("Enter sides a, b, c: ", 3, &a, &b, &c)) {
        puts("Invalid input.");
        return;
    }

    printf("a = %.4f, b = %.4f, c = %.4f\n", a, b, c);
    printf("Angle gamma: %.4f deg\n", triangle_angle_gamma_deg(a, b, c));
    printf("Bisector w_g: %.4f\n", triangle_bisector_gamma(a, b, c));
}
```

1.3. Program results

=== Task 1: triangle - angle gamma and bisector w ===

Enter sides a, b, c: 38.93 42.21 34.42

a = 38.9300, b = 42.2100, c = 34.4200

Angle gamma: 50.0002 deg

Bisector w_g: 36.7088

Fig. 3 – Task 1 results

Task 2

Develop the calculation algorithm and the C language program to determine the values of the function:

$$F = \frac{E1^2 + |E2| + E3}{\sqrt{|E1 \cdot E3|}} - \frac{1}{3}, \quad (1)$$

The functions E1, E2, E3 according to the options are given in table 2.1. Output of source data should be carried out with the appropriate explanatory text and formatted output.

2.1. Block diagram of the program algorithm

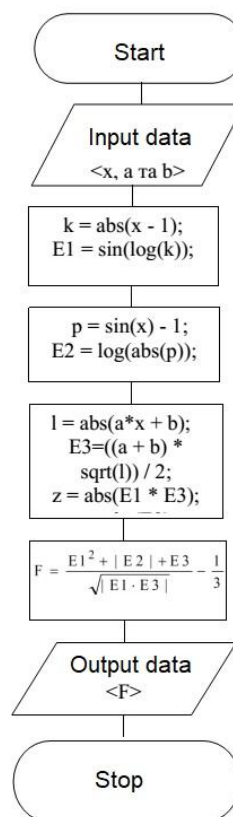


Fig. 4 - Block diagram of the program algorithm

2.2. Program text

```
#include "tasks.h"
#include "utils.h"
#include <math.h>
#include <stdio.h>

static double evaluate(const double a, const double b, const double x) {
    const double e1 = cbrt(fabs(a * x + b - 3.0));
    const double e2 = cbrt(fabs(b * x - b + 4.0));
    const double e3 = exp(-b * x / (a + 3.0));

    return (e1 * e1 + fabs(e2) + e3) / sqrt(fabs(e1 * e3)) - 1.0 / 3.0;
}

void run_task2(void) {
    double a = 0.0;
    double b = 0.0;
    double x = 0.0;

    puts(
        "\n=== Task 2: function F(E1, E2, E3) ===\n\n"
        "Formulas:\n"
        " - E1 = cbrt(|a*x + b - 3|)\n"
        " - E2 = cbrt(|b*x - b + 4|)\n"
        " - E3 = exp(-b*x / (a + 3))\n"
        " - F = (E1^2 + |E2| + E3) / sqrt(|E1 * E3|) - 1/3\n"
    );

    if (!read_doubles("Enter a, b, x: ", 3, &a, &b, &x)) {
        puts("Invalid input.");
        return;
    }
    if (a == -3.0) {
        puts("Error: 'a = -3' leads to division by zero in E3.");
        return;
    }
    if (a * x + b - 3.0 == 0.0) {
        puts("Error: E1 is 0, leading to division by zero in F.");
        return;
    }

    printf("\na = %.4f, b = %.4f, x = %.4f\n", a, b, x);
    printf("F = %.4f\n", evaluate(a, b, x));
}
```

1.3. Program results

=== Task 2: function F(E1, E2, E3) ===

Formulas:

- $E1 = \text{cbrt}(|a*x + b - 3|)$
- $E2 = \text{cbrt}(|b*x - b + 4|)$
- $E3 = \exp(-b*x / (a + 3))$
- $F = (E1^2 + |E2| + E3) / \text{sqrt}(|E1 * E3|) - 1/3$

Enter a, b, x: 1 1 1

a = 1.0000, b = 1.0000, x = 1.0000
F = 3.4811

Fig. 5 – Task 2 results

Task 3

By using Horner's algorithm, compile a program to calculate the value of the polynomial. According to the compiled program to calculate the value of the polynomial at a given value X. Output of the original data to carry out with the appropriate explanatory and formatted output. Task options are given in table 3.1.

3.1. Block diagram of the program algorithm

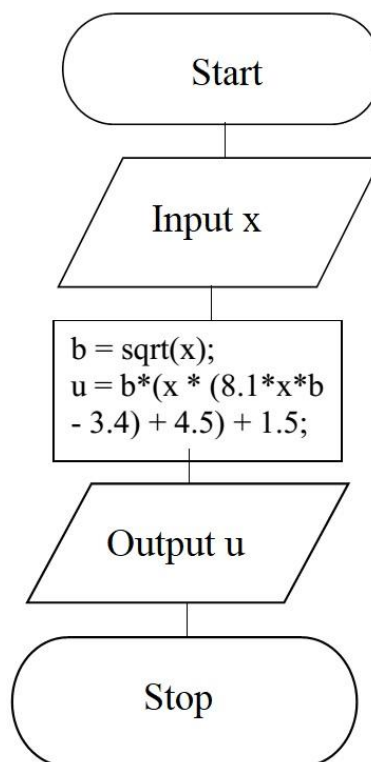


Fig. 6 – Block diagram of the program algorithm

3.2. Program text

```
#include "tasks.h"
#include "tasks.h"
#include "utils.h"

#include <math.h>
#include <stdio.h>

static double horner_evaluate(
    const double *coefficients,
    const int degree,
    const double x
) {
    double result = coefficients[0];

    for (int i = 1; i <= degree; ++i) {
        result = result * x + coefficients[i];
    }

    return result;
}

static double evaluate(const double x) {
    const double coefficients[] = {-3.7, 0.0, 0.0, 4.1, 0.0, -2.1, 1.2};
    const int degree = 6;
    const double u = 1.0 + sqrt(x);

    return horner_evaluate(coefficients, degree, u);
}

void run_task3(void) {
    double x = 0.0;

    puts(
        "\n=== Task 3: Horner's method ===\n\n"
        "Formula:  $c = -3.7 \cdot (1 + \sqrt{x})^6 + 4.1 \cdot (1 + \sqrt{x})^3 - 2.1 \cdot (1 + \sqrt{x}) + 1.2$ \n\n"
    );

    if (!read_double("Enter x: ", &x)) {
        puts("Invalid input.");
        return;
    }
}
```

```

if (x < 0.0) {
    puts("x must be >= 0 - skipping.");
    return;
}

printf("\nx = %.4f\n", x);
printf("c = %.4f\n", evaluate(x));
}

```

3.3. Program results

=== Task 3: Horner's method ===

Formula: $c = -3.7 \cdot (1 + \sqrt{x})^6 + 4.1 \cdot (1 + \sqrt{x})^3 - 2.1 \cdot (1 + \sqrt{x}) + 1.2$

Enter x: 30.5

x = 30.5000

c = -283817.9510

Fig. 7 – Task 3 results

Task 4

Find the area of the shaded figure. Output the output data with the appropriate explanatory text and formatted output.

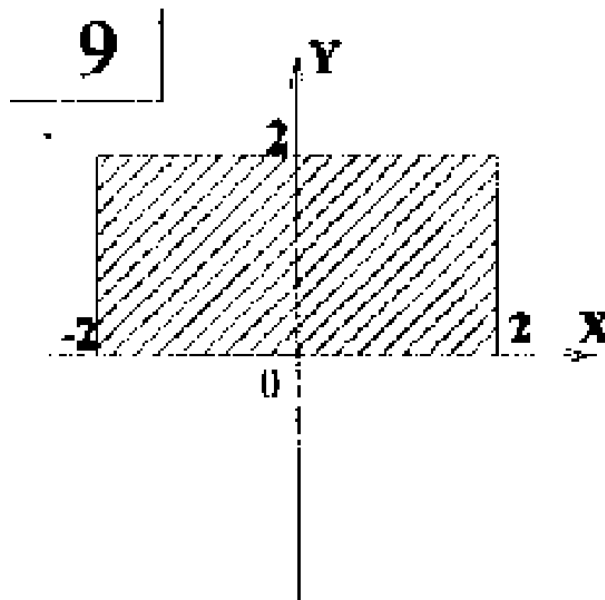


Fig. 8 – The shaded figure

4.1. Block diagram of the program algorithm

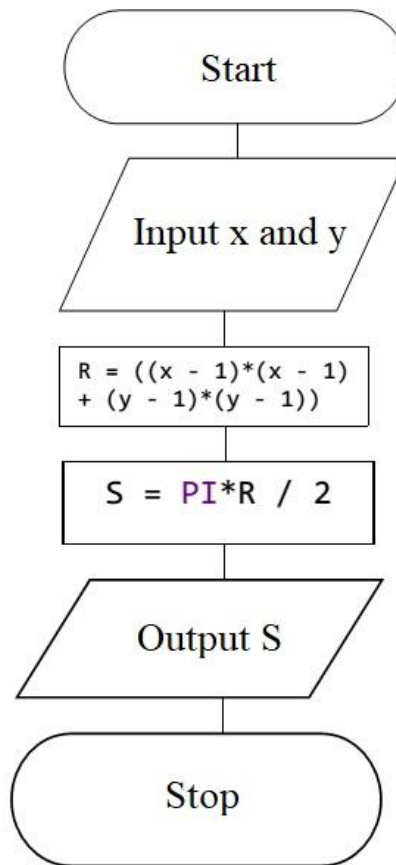


Fig. 9 – Block diagram of the program algorithm

4.2. Program text

```
#include "tasks.h"
#include "utils.h"
#include <math.h>
#include <stdio.h>

void run_task4(void) {
    double coords[4] = {0.0};

    puts("\n=== Task 4: Area of an axis-aligned rectangle ===\n");

    if (!read_doubles(
        "Enter x1 y1 x2 y2: ",
        4,
        &coords[0],
        &coords[1],
        &coords[2],
        &coords[3]
    )) {
        puts("Invalid input.");
    }
}
```

```

    return;
}

const double x1 = coords[0];
const double y1 = coords[1];
const double x2 = coords[2];
const double y2 = coords[3];

const double width = fabs(x2 - x1);
const double height = fabs(y2 - y1);
const double area = width * height;

if (area == 0.0) {
    puts(
        "Warning: Area is 0.0 (the points form a line segment or a point).";
    );
}

printf("\nCorners:\n");
printf(" - 1st corner: (%.4f, %.4f)\n", x1, y1);
printf(" - 2nd corner: (%.4f, %.4f)\n", x2, y2);

printf("\nDimensions:\n");
printf(" - Width: %.4f\n", width);
printf(" - Height: %.4f\n", height);

printf("\nArea: %.4f\n", area);
}

```

4.3. Program results

=== Task 4: Area of an axis-aligned rectangle ===

Enter x1 y1 x2 y2: -2 0 2 2

Corners:

- 1st corner: (-2.0000, 0.0000)
- 2nd corner: (2.0000, 2.0000)

Dimensions:

- Width: 4.0000
- Height: 2.0000

Area: 8.0000

Fig. 10 – Task 4 results

Conclusions

- 1) We studied the structure of the C language program.
- 2) MS Visual Studio programming environment has been studied.
- 3) We studied of the I/O operators and simple data types in the program, compiling the simplest block diagrams of programs and programs in C language.

APPENDIX A

Source code of the “tasks.h” file

```
#ifndef TASKS_H
#define TASKS_H

#define TASK_LIST X(1) X(2) X(3) X(4)

#define X(num) void run_task##num(void);
TASK_LIST
#undef X

#define X(num) +1
enum { TASK_COUNT = 0 TASK_LIST };
#undef X

void run_task(int n);
void run_all_tasks(void);

#endif // TASKS_H
```

APPENDIX B

Source code of the “utils.h” file

```
#ifndef UTILS_H
```

```
#define UTILS_H
```

```
#include <stdbool.h>
```

```
#include <stddef.h>
```

```
bool read_double(const char *prompt, double *out_value);
```

```
bool read_doubles(const char *prompt, size_t count, ...);
```

```
#endif // UTILS_H
```

```
#ifndef PI
```

```
#define PI 3.14159265358979323846
```

```
#endif // PI
```

APPENDIX C

Source code of the “main.c” file

```
#include "tasks.h"

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void) {
    printf(
        "=====\\n"
        " Laboratory work 1: Linear algorithms (variant 9)\\n"
        "=====\\n\\n"
        "Enter task number (1-%d) or press Enter to run all tasks [ALL]: ",
        TASK_COUNT
    );

    char input[32] = "";
    if (fgets(input, sizeof(input), stdin) != NULL) {
        input[strcspn(input, "\\r\\n")] = '\\0';
    }

    char *end;
    const long task = strtol(input, &end, 10);
    int status = 0;

    if (input[0] == '\\0') {
        run_all_tasks();
    } else if (*end == '\\0' && task >= 1 && task <= TASK_COUNT) {
        run_task((int)task);
    } else {
        printf("Error: invalid task '%s'.\\n", input);
        status = 1;
    }

    printf("\\nPress Enter to exit...");
    getchar();

    return status;
}
```

APPENDIX D

Source code of the “runner.c” file

```
#include "tasks.h"

static void (*const TASKS[TASK_COUNT])(void) = {
#define X(num) run_task##num,
    TASK_LIST
#undef X
};

void run_task(const int n) {
    TASKS[n - 1]();
}

void run_all_tasks(void) {
    for (int i = 1; i <= TASK_COUNT; ++i) {
        run_task(i);
    }
}
```

APPENDIX E

Source code of the “utils.c” file

```
#include "utils.h"

#include <ctype.h>
#include <errno.h>
#include <stdarg.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

static bool is_buffer_empty(const char *ptr) {
    while (*ptr != '\0') {
        if (!isspace((unsigned char)*ptr)) {
            return false;
        }

        ptr++;
    }

    return true;
}

bool read_double(const char *prompt, double *out_value) {
    return read_doubles(prompt, 1, out_value);
}

bool read_doubles(const char *prompt, size_t count, ...) {
    if (count == 0) {
        return false;
    }

    if (prompt != NULL) {
        printf("%s", prompt);
    }

    char buffer[256];
    if (fgets(buffer, sizeof(buffer), stdin) == NULL) {
        return false;
    }

    buffer[strcspn(buffer, "\r\n")] = '\0';

    va_list args;
```

```

va_start(args, count);

const char *ptr = buffer;

for (size_t i = 0; i < count; ++i) {
    double *out = va_arg(args, double *);

    if (out == NULL) {
        va_end(args);
        return false;
    }

    char *endptr;
    errno = 0;
    *out = strtod(ptr, &endptr);

    if (ptr == endptr || errno != 0) {
        va_end(args);

        return false;
    }

    ptr = endptr;
}

va_end(args);

return is_buffer_empty(ptr);
}

```